

词嵌入与NLP

学习目标

在RNN中使用one_hot表示的问题

词嵌入

特点:

算法类别

Word2Vec案例

训练语料

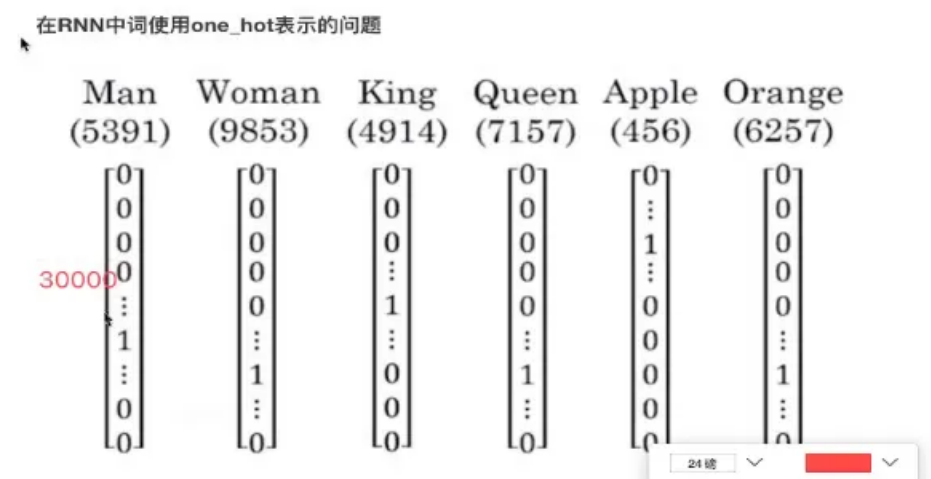
步骤

代码

学习目标

- 目标
 - 了解词嵌入的优势
 - 掌握词嵌入的训练gensim库的使用
- 应用
 - 无

在RNN中使用one_hot表示的问题



假设有10000个词

每个词的向量长度都为10000，整体大小太大

没能表示出词与词之间的关系

例如Apple与Orange会更近一些，Man与Woman会近一些，取任意两个向量计算内积都为0

词嵌入

定义：指把一个维度为所有词的数量的高维空间嵌入到一个维数低得很多得连续向量空间中，每个单词或词组被映射为实数域上得向量

	Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)
Gender	-1	1	-0.95	0.97	0.00	0.01
Royal	0.01	0.02	0.93	0.95	-0.01	0.00
Age	0.03	0.02	0.7	0.69	0.03	-0.02
Food	0.09	0.01	0.02	0.01	0.95	0.97

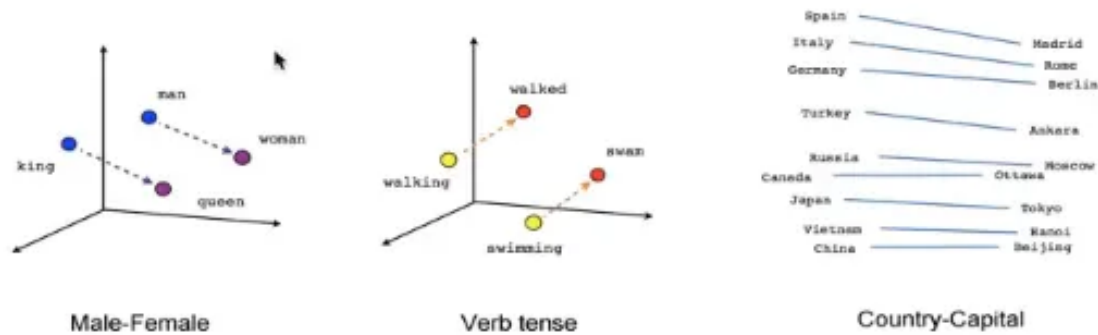
特点：

能够体现出词与词之间的关系

比如说我们用man-woman，或者apple-orange，都能得到一个向量

能够得到相似词，例如man-woman=king-？

？=Queen



算法类别

Bengio等人在一系列论文中使用了神经概率语言模型使机器“习得词语的分布式表示”

2013年，谷歌托马斯·米科洛维 (Tomas Mikolov) 领导的团队发明了一套工具word2vec来进行词嵌入。

- skip-gram

算法学习实现：<https://www.tensorflow.org/tutorials/representation/word2vec>

算法实现：

<https://www.tensorflow.org/tutorials/representation/word2vec>

- CBow

下载gensim库

```
pip install gensim
```

Word2Vec案例

训练语料

下载地址：<http://sougou.com/labs/resource/cs.php>

该地址已经停止访问，后续会通过截图表示该部分

搜狗新闻中文语料（2.7G）

做中文分词处理之后的结果

步骤

训练模型

测试模型结果

代码

训练模型API

- 训练模型API
 - `from gensim import Word2Vec`
 - `Word2Vec(LineSentence(inp), size=400, window=5, min_count=5)`
 - `LineSentence(inp)`: 把word2vec训练模型的磁盘存储文件
 - 转换成所需要的格式,如: `[["sentence1"],["sentence1"]]`
 - `size`: 是每个词的向量维度
 - `window`: 是词向量训练时的上下文扫描窗口大小, 窗口为5就是考虑前5个词和后5个词
 - `min-count`: 设置最低频率, 默认是5, 如果一个词语在文档中出现的次数小于5, 那么就会丢弃
 - 方法:
 - `inp`: 分词后的文本
 - `save(outp1)`: 保存模型

训练的代码如下

```
if len(sys.argv) < 3:
    sys.exit(1)

# inp表示语料库(分词), outp: 模型
inp, outp = sys.argv[1:3]

model = Word2Vec(LineSentence(inp), size=400, window=5, min_count=5, workers=multiproc
model.save(outp)
```

```
import sys
import multiprocessing

from gensim.models import Word2Vec
from gensim.models.word2vec import LineSentence

if __name__ == '__main__':

    if len(sys.argv) < 3:
        sys.exit(1)

    # inp表示语料库(分词), outp: 模型
    inp, outp = sys.argv[1:3]

    model = Word2Vec(LineSentence(inp), size=400, window=5, min_count=5, workers=multiproc
    model.save(outp)
```

运行命令

```
python trainword2vec.py ./corpus_seg.txt ./model/*
```

制定好分词的文件以及, 保存模型的文件

加载模型测试代码

- `model = gensim.models.Word2Vec.load("*.model")`
 - `model.most_similar('警察')`
 - `model.similarity('男人','女人')`
 - `most_similar(positive=['女人', '丈夫'], negative=['男人'], topn=1)`

```
import gensim
gensim.models.Word2Vec.load("./model/corpus.model")

model.most_similar("警察")
```

Out:

```
[('警员', 0.6961891651153564),
 ('保安人员', 0.6414757370948792),
 ('警官', 0.6149201989173889),
 ('消防员', 0.6082159876823425),
 ('宪兵', 0.6013336181640625),
 ('保安', 0.5982533693313599),
 ('武警战士', 0.5962344408035278),
 ('公安人员', 0.5880240201950073),
 ('民警', 0.5878666639328003),
 ('刑警', 0.5800305604934692)]
```

```
model.similarity('男人','女人')
```

Out: 0.8909852730435042

```
model.most_similar(positive=['女人', '丈夫'], negative=['男人'], topn=1)
```

Out: [('妻子', 0.7788498997688293)]

